

CoachPro

High-Level Design Document

MVP — Feature-by-Feature Build Guide

Version	v1.0
Stack	Next.js · TypeScript · Shadcn UI · Tailwind · Supabase · Vercel
Date	June 2026
Status	Draft — Engineering Review
Scope	MVP Core (v1.0) — 8 Feature Modules

Confidential · For Internal Engineering Use Only

1. Document Overview & Build Philosophy

This HLD defines the system architecture, data model, API contracts, and component structure for CoachPro MVP (v1.0). It is organised feature-by-feature in the recommended build sequence — each module is self-contained enough to be developed, tested, and deployed independently.

1.1 Tech Stack

Layer	Technology	Role
Frontend	Next.js 14 (App Router) + TypeScript	Server/client components, routing, SSR
Styling	Tailwind CSS + shadcn/ui	Utility-first UI, accessible components
Database	Supabase PostgreSQL	Primary data store, RLS enforcement
Auth	Supabase Auth	Email/password, magic link, JWT sessions
Storage	Supabase Storage	Profile photos, receipt PDFs, logos
Security	Row Level Security (RLS)	Tenant isolation at database layer
Hosting	Vercel	CI/CD, edge functions, preview deploys
SMS	MSG91 / Twilio (pluggable)	Fee reminders, notifications

1.2 Core Architectural Principles

- Tenant isolation via RLS: every table has a `tenant_id` column; RLS policies enforce data separation at the DB layer — no application-level tenant filtering required.
- Auth → Role → Route: after login, the system reads the user's role from the profiles table and routes them to the correct dashboard. No role-selector UI.
- Session as a first-class entity: batch occurrences and 1-to-1 sessions share common structure, enabling attendance, notes, and conflict detection to work uniformly.
- Money in paise: all monetary values stored as integers in paise (1 INR = 100 paise) to eliminate floating-point errors in fee calculations.
- Storage via Supabase buckets: photos, logos, and PDFs stored in named buckets; public or signed URLs served to clients.

1.3 Build Order (Module Sequence)

Why this order?

Each module depends on the previous. You cannot create batches without coaches; you cannot generate fee ledgers without students in batches. Follow this sequence to avoid blockers.

#	Module	Key Deliverable	Unlocks
1	Foundation & Auth	Supabase project, RLS, roles, login/signup	Everything
2	Tenant & Academy Setup	Onboarding wizard, academy profile, subdomain	Modules 3–8
3	Coach Management	Coach CRUD, invite email, coach dashboard	Batch creation
4	Student Management	Student CRUD, CSV import, jersey fields	Fee ledger
5	Batch Management	Batch CRUD, conflict detection, capacity tracking	Calendar, Fees
6	Calendar & Scheduling	Day/Week/Month views, drag-drop, 1-to-1 booking	SMS triggers
7	Fee Management	Ledger, payments, receipts, dashboard	SMS, Reports
8	SMS Notifications	Manual SMS, templates, delivery tracking	Phase 2 auto-SMS

2. Module 1 — Foundation & Auth

This is the bedrock of the entire platform. Nothing else can be built without this. All subsequent modules assume this module is complete and stable.

2.1 Objectives

- Create and configure the Supabase project (database, auth, storage buckets).
- Define the multi-tenant schema with RLS policies enforcing tenant isolation.
- Implement role-based routing: Admin lands on /dashboard, Coach on /coach.
- Build the login, signup, and magic-link pages in Next.js.

2.2 Database — Core Tables

tenants		
Column	Type / Constraint	Notes
id	UUID PK	Auto-generated on academy signup
name	TEXT NOT NULL	Academy display name
subdomain	TEXT UNIQUE NOT NULL	e.g. suncricket — used for routing
logo_url	TEXT	Supabase Storage public URL
sports	TEXT[]	Array of sport names offered
timezone	TEXT DEFAULT 'Asia/Kolkata'	For display & scheduling
plan	TEXT DEFAULT 'free'	free starter growth pro enterprise
sms_credits	INT DEFAULT 0	Remaining SMS credits
working_hours	JSONB	{ mon: {open:"06:00", close:"20:00"}, ... }
created_at	TIMESTAMPTZ DEFAULT now()	

profiles		
Column	Type / Constraint	Notes
id	UUID PK → auth.users.id	Mirrors Supabase Auth user
tenant_id	UUID FK → tenants.id	RLS key — set on every row
role	TEXT NOT NULL	admin coach parent super_admin
full_name	TEXT NOT NULL	
mobile	TEXT	E.164 format for SMS

profiles		
Column	Type / Constraint	Notes
avatar_url	TEXT	Supabase Storage URL
status	TEXT DEFAULT 'active'	active suspended
created_at	TIMESTAMPTZ DEFAULT now()	

2.3 RLS Policy Pattern

Critical Rule

Every table in the CoachPro schema must have tenant_id and the RLS policies below. This enforces tenant isolation at the database layer, removing any risk of application-level bugs leaking data.

Apply this policy pattern to every table:

```
ALTER TABLE <table> ENABLE ROW LEVEL SECURITY; CREATE POLICY tenant_isolation ON <table> USING (tenant_id = (SELECT tenant_id FROM profiles WHERE id = auth.uid()));
```

2.4 Auth Flow

Step	Actor	Action	System Response
1	Visitor	Hits coachpro.app → clicks 'Start Free Trial'	Shows signup form
2	Visitor	Enters academy name, email, mobile, sport	Supabase Auth creates user + triggers DB function
3	System	DB function: creates tenants row + profiles row (admin)	Sets role = 'admin', assigns subdomain
4	Admin	Redirected to /onboarding wizard	Session JWT contains tenant_id + role
5	Coach	Receives invite email, sets password	Profile created with role = 'coach'
6	Any user	Logs in → JWT decoded → role read from profiles	Routed to /dashboard (admin) or /coach (coach)

2.5 Next.js App Structure

Route layout for the Next.js App Router:

- app/(auth)/login, /signup, /invite/[token] — public auth routes
- app/(admin)/dashboard, /students, /coaches, /batches, /calendar, /fees, /settings — Admin layout with sidebar

- `app/(coach)/coach/calendar, /coach/students` — Coach-scoped layout
- `middleware.ts` — reads JWT, checks role, enforces route protection

2.6 Supabase Storage Buckets

Bucket	Access	Contents
academy-logos	Public	Tenant logo images
profile-photos	Signed URL	Coach & student photos
receipts	Signed URL	Generated PDF receipts

3. Module 2 — Tenant & Academy Setup

This module covers everything an admin configures before adding coaches and students. It includes the onboarding wizard and the Settings area. Completed immediately after Module 1.

3.1 Objectives

- Build the onboarding wizard: 4-step guided setup flow after first login.
- Build the Settings page: editable academy profile, working hours, fee currency, payment modes.
- Implement subdomain routing: middleware reads subdomain and injects tenant context.
- Implement subscription plan gating: enforce Free tier limits (15 students, 2 batches, 1 coach).

3.2 Onboarding Wizard — Steps

Step	Title	Fields / Actions	Skip-able?
1	Academy Profile	Academy name, sport(s), address, contact, logo upload, timezone	No
2	Add First Coach	Name, email, mobile — sends invite email immediately	Yes
3	Create First Batch	Batch name, sport, days, time, venue, monthly fee	Yes
4	Enrol First Student	Name, DOB, guardian mobile, assign to batch	Yes

3.3 Settings — Academy Configuration

- Academy Profile: name, logo, sports offered, address, contact number, timezone.
- Working Hours: per-day open/close time grid. Stored as JSONB in tenants table. Used to constrain batch scheduling.
- Fee Settings: default currency (INR for v1.0), accepted payment modes checkboxes (Cash / UPI / Card / Cheque).
- SMS Settings: sender name/ID, opt-out management, view remaining credits.
- Subscription: current plan display, upgrade CTA, student/batch/coach counts vs. plan limits.

3.4 Subdomain Routing

Each tenant gets a subdomain (e.g., greenfield.coachpro.app). The Next.js middleware reads the Host header, strips the subdomain, looks up the tenants table, and injects the `tenant_id` into the request context. All subsequent Supabase calls automatically include `tenant_id`.

Free Tier Enforcement

A planGuard() server-side utility checks the tenant's current plan against hard limits before any create/update operation. Returns a 403 with an upgrade prompt if the limit is reached.

4. Module 3 — Coach Management

Coaches are the second actor in the system. This module creates coach accounts, sends invite emails, and builds the coach-facing dashboard. Must be completed before batches can be assigned.

4.1 Objectives

- Admin creates a coach profile: name, photo, mobile, email, sport(s), availability, status.
- System sends an email invitation via Supabase Auth; coach sets their password.
- Build the Coach dashboard: personal calendar, assigned batch list, student list.
- Enforce plan limits: Free = 1 coach, Starter = 3, Growth = 10, Pro = unlimited.

4.2 Database

coaches		
Column	Type / Constraint	Notes
id	UUID PK	Same as profiles.id
tenant_id	UUID FK → tenants.id	RLS key
profile_id	UUID FK → profiles.id	1-to-1 with profiles
sports	TEXT[]	Sports this coach is qualified for
availability	JSONB	{ mon: [{start:"08:00", end:"12:00"}], ... }
status	TEXT DEFAULT 'active'	active inactive
joined_at	DATE	Start date for payroll (Phase 3)

4.3 API Routes (Next.js Route Handlers)

Method	Route	Auth	Description
GET	/api/coaches	Admin	List all coaches for tenant
POST	/api/coaches	Admin	Create coach + send invite email
PATCH	/api/coaches/[id]	Admin	Update coach profile / availability
DELETE	/api/coaches/[id]	Admin	Soft-deactivate (status = inactive)
GET	/api/coaches/[id]/schedule	Admin/Coach	Fetch this coach's sessions

4.4 Coach Dashboard (Role-Gated View)

- My Calendar: Week view defaulting to current week. Only own sessions visible.
- My Batches: list of assigned batches with student count, schedule summary.
- My Students: aggregated student list across all assigned batches. Search by name.
- 1-to-1 Sessions: upcoming private sessions in a card list format.

5. Module 4 — Student Management

Students are the primary entity around which all features revolve. This module must be completed before the fee ledger and attendance features can function.

5.1 Objectives

- Admin creates and manages student profiles with required and optional fields.
- Support CSV bulk import for migrating academies.
- Student profile page aggregates: batch assignments, fee ledger, session history, jersey details.
- Search and filter: by name, batch, sport, calling name, fee status.

5.2 Database

students		
Column	Type / Constraint	Notes
id	UUID PK	Auto-generated
tenant_id	UUID FK → tenants.id	RLS key
full_name	TEXT NOT NULL	
calling_name	TEXT	Preferred informal name (e.g. Chintu)
dob	DATE NOT NULL	
gender	TEXT NOT NULL	Male Female Other
guardian_name	TEXT NOT NULL	
guardian_mobile	TEXT NOT NULL	E.164 for SMS
guardian_email	TEXT	
sports	TEXT[]	Sports enrolled in
enrolment_date	DATE NOT NULL	
status	TEXT DEFAULT 'active'	active inactive suspended
photo_url	TEXT	Supabase Storage URL
jersey_size	TEXT	XS S M L XL XXL
jersey_number	INT	Favourite jersey number
jersey_name	TEXT	Name on back of jersey; defaults to calling_name
sms_opt_in	BOOL DEFAULT true	Guardian's SMS opt-in status

students		
Column	Type / Constraint	Notes
created_at	TIMESTAMPTZ DEFAULT now()	

batch_students (junction)		
Column	Type / Constraint	Notes
id	UUID PK	
tenant_id	UUID FK → tenants.id	RLS key
batch_id	UUID FK → batches.id	
student_id	UUID FK → students.id	
enrolled_at	DATE NOT NULL	
status	TEXT DEFAULT 'active'	active waitlisted dropped

5.3 API Routes

Method	Route	Description
GET	/api/students	List students with filters (batch, sport, status)
POST	/api/students	Create student + optionally assign to batches
GET	/api/students/[id]	Student profile with batch, fee, and session summary
PATCH	/api/students/[id]	Update student fields
POST	/api/students/import	CSV bulk import — validate, deduplicate, insert
POST	/api/students/[id]/batches	Assign student to additional batch

5.4 CSV Import

- Template: full_name, dob, gender, guardian_name, guardian_mobile, sports, jersey_size, jersey_number — downloadable from the import page.
- Validation: duplicate mobile detection, date format checking, enum validation for jersey size.
- Preview step: admin reviews a parsed table before confirming import.
- Error report: rows with issues flagged with specific error messages; valid rows imported regardless.

6. Module 5 — Batch Management

A Batch is a recurring group session — the core scheduling unit of the platform. Conflict detection, capacity tracking, and the fee system all depend on this module.

6.1 Objectives

- Admin creates batches with recurring schedule, coach, venue, capacity, and fee.
- Conflict detection: prevent double-booking of coaches and venues.
- Capacity tracking with visual indicator and waiting list.
- Batch lifecycle: Active → Paused → Closed (soft-archived).

6.2 Database

batches		
Column	Type / Constraint	Notes
id	UUID PK	
tenant_id	UUID FK → tenants.id	RLS key
name	TEXT NOT NULL	e.g. 'U-12 Cricket Morning'
sport	TEXT NOT NULL	
coach_id	UUID FK → coaches.id	Primary coach
days_of_week	INT[]	0=Sun ... 6=Sat
start_time	TIME NOT NULL	
end_time	TIME NOT NULL	
venue	TEXT	Venue / court name
capacity	INT	Max students; NULL = unlimited
monthly_fee	INT NOT NULL	In paise (100 = ₹1)
status	TEXT DEFAULT 'active'	active paused closed
effective_from	DATE NOT NULL	When the batch starts running
created_at	TIMESTAMP TZ DEFAULT now()	

6.3 Conflict Detection Logic

Algorithm

Before saving a batch, run: `SELECT COUNT(*) FROM batches WHERE coach_id = $coach AND status = 'active' AND days_of_week && $days AND NOT (end_time <= $start OR start_time >= $end)`. If count > 0, return conflict warning with the clashing batch details. Same check for venue conflicts.

6.4 Capacity & Waiting List

- Enrolled count = `SELECT COUNT(*) FROM batch_students WHERE batch_id = $id AND status = 'active'`.
- When enrolled $\geq$ capacity: new students are added with status = 'waitlisted'.
- When an active student is removed: first waitlisted student is automatically promoted to 'active' and admin is notified.
- UI indicator: green (< 80%), amber (80–99%), red (full) badge on batch card.

6.5 API Routes

Meth od	Route	Description
GET	/api/batches	List all batches for tenant
POST	/api/batches	Create batch with conflict check
PATCH	/api/batches/[id]	Update batch; re-run conflict check if schedule changed
GET	/api/batches/[id]/students	List enrolled students for batch
POST	/api/batches/[id]/students	Enrol student (or add to waiting list)
DELETE	/api/batches/[id]/students/[sid]	Remove student; promote next waitlisted

7. Module 6 — Calendar & Scheduling

The Calendar is the primary operational surface of CoachPro. All session management — viewing, creating, rescheduling, and booking 1-to-1 sessions — happens here.

7.1 Objectives

- Render Day / Week / Month views with all batch occurrences and 1-to-1 sessions.
- Admin view: all coaches, colour-coded. Coach view: own sessions only.
- 1-to-1 session booking with double-booking prevention.
- Drag-and-drop rescheduling for admins (conflict validation before save).
- Session detail card: click to see participants, venue, status, notes.

7.2 Database

sessions (1-to-1)		
Column	Type / Constraint	Notes
id	UUID PK	
tenant_id	UUID FK → tenants.id	RLS key
coach_id	UUID FK → coaches.id	
student_id	UUID FK → students.id	
date	DATE NOT NULL	
start_time	TIME NOT NULL	
end_time	TIME NOT NULL	
venue	TEXT	
fee_override	INT	Paise; NULL uses tenant default
status	TEXT DEFAULT 'scheduled'	scheduled completed cancelled no_show
notes	TEXT	Post-session coach feedback
created_at	TIMESTAMPTZ DEFAULT now()	

7.3 Calendar Data Query Pattern

Batch occurrences are computed on-the-fly (not stored as individual rows until Phase 2 attendance). The API generates occurrence dates for the requested date range from the batch's days_of_week, start_time, and effective_from:

- GET /api/calendar?from=2026-06-01&to=2026-06-30 → returns {batch_occurrences: [...], sessions: [...]}

- Each batch_occurrence includes: batch_id, date, start_time, end_time, venue, coach, enrolled_count.
- Each session is a direct query from the sessions table.

7.4 Frontend Calendar Component

View	Library / Approach	Notes
Month	Custom grid or react-big-calendar	Dot indicators per day; click to expand
Week	react-big-calendar or FullCalendar	Colour-coded lanes per coach
Day	Time-slot grid (custom)	30-min slots; drag handle for admin

Colour Legend

Each coach is assigned a colour from a tenant-defined palette (stored in coaches table as hex string). Batch events use the coach colour. 1-to-1 sessions use a slightly darker shade. Unassigned slots are grey.

7.5 1-to-1 Booking Flow

- Admin or coach opens 'New 1-to-1' modal: selects coach, student, date, start/end time, venue, optional fee.
- On submit: API checks for coach conflicts (existing sessions + batch occurrences at that time) and student conflicts.
- If clear: session inserted; calendar refreshes. If conflict: specific clash returned to UI.
- Status transitions: Scheduled → (coach action) Completed or No-show, or (admin) Cancelled.

8. Module 7 — Fee Management

Fee management is the highest-value feature for academy admins. It covers the monthly ledger, payment recording, receipt generation, and the fee dashboard with overdue tracking.

8.1 Objectives

- Auto-generate monthly fee ledger entries for all active students in active batches.
- Admin records payments (full or partial) with payment mode and receipt generation.
- Fee dashboard: total collected, total outstanding, overdue count, collection rate %.
- PDF receipt auto-generated on payment; downloadable and shareable.
- One-click SMS trigger to guardian from the defaulters list.

8.2 Database

fee_ledger		
Column	Type / Constraint	Notes
id	UUID PK	
tenant_id	UUID FK → tenants.id	RLS key
student_id	UUID FK → students.id	
batch_id	UUID FK → batches.id	NULL for 1-to-1 fees
month_year	DATE	First day of the month (2026-06-01)
amount_due	INT NOT NULL	Paise; prorated on mid-month join
amount_paid	INT DEFAULT 0	Paise; updated on each payment
balance	INT GENERATED	amount_due - amount_paid (computed)
status	TEXT DEFAULT 'pending'	pending partial paid waived
due_date	DATE	Configurable per tenant (e.g. 5th of month)
notes	TEXT	Admin notes on waived/adjusted fees

fee_payments		
Column	Type / Constraint	Notes
id	UUID PK	
tenant_id	UUID FK → tenants.id	RLS key

fee_payments		
Column	Type / Constraint	Notes
ledger_id	UUID FK → fee_ledger.id	
amount	INT NOT NULL	Paise
paid_at	TIMESTAMPTZ NOT NULL	
payment_mode	TEXT NOT NULL	cash upi card cheque
receipt_number	TEXT UNIQUE	Auto-generated sequential per tenant
receipt_url	TEXT	Supabase Storage URL for PDF
recorded_by	UUID FK → profiles.id	Admin who recorded the payment
voided_at	TIMESTAMPTZ	Set if payment is reversed

8.3 Ledger Auto-Generation

Scheduled Job

A Supabase Edge Function (or Vercel Cron) runs on the 1st of each month. It selects all active batch_students for active batches and inserts fee_ledger rows for the new month. If a row already exists (idempotent), it skips. Prorated amounts are calculated when enrolment_date is within the current month.

8.4 Receipt Generation

- On payment record: a server-side function generates a PDF using a HTML template.
- PDF includes: academy logo, receipt number, student name, amount, date, payment mode, admin name.
- PDF uploaded to Supabase Storage bucket 'receipts' with path: tenant_id/year/month/receipt_number.pdf.
- Signed URL returned to client (valid 7 days); stored in fee_payments.receipt_url.

8.5 Fee Dashboard Queries

Card	Query	Notes
Total Collected	SUM(amount_paid) WHERE month_year = current_month	Real-time from fee_payments
Total Outstanding	SUM(balance) WHERE status IN ('pending','partial')	Current month only
Overdue Count	COUNT(*) WHERE due_date < today AND status != 'paid'	Cross-month view
Collection Rate	SUM(paid) / SUM(due) * 100	Current month %

8.6 API Routes

Method	Route	Description
GET	/api/fees	Fee dashboard summary (cards + defaulters list)
GET	/api/fees/ledger	Paginated ledger with filters
POST	/api/fees/ledger/generate	Trigger manual ledger generation (admin)
POST	/api/fees/payments	Record a payment; triggers receipt generation
DELETE	/api/fees/payments/[id]	Void a payment (soft-delete)
GET	/api/fees/receipts/[id]	Get signed URL for receipt PDF

9. Module 8 — SMS Notifications

v1.0 ships with manual SMS capability only. The architecture is built to support automated triggers in Phase 2 without rewrites. SMS delivery is pluggable — the gateway can be swapped per tenant.

9.1 Objectives

- Admin sends a manual SMS to any guardian from the fee dashboard (single or bulk).
- Message templates with placeholders: {guardian_name}, {student_name}, {amount_due}, {month}, {academy_name}.
- Delivery status logged against each ledger entry: Sent / Delivered / Failed.
- Per-student SMS opt-out respected (sms_opt_in field on students table).
- Credit balance deducted on send; admin alerted at 20 remaining credits.

9.2 Database

sms_logs		
Column	Type / Constraint	Notes
id	UUID PK	
tenant_id	UUID FK → tenants.id	RLS key
ledger_id	UUID FK → fee_ledger.id	NULL for non-fee SMS
student_id	UUID FK → students.id	
mobile	TEXT NOT NULL	Recipient number (E.164)
message	TEXT NOT NULL	Resolved message text
gateway_ref	TEXT	External message ID from gateway
status	TEXT DEFAULT 'sent'	sent delivered failed
sent_at	TIMESTAMPTZ DEFAULT now()	
delivered_at	TIMESTAMPTZ	Set via gateway webhook

sms_templates		
Column	Type / Constraint	Notes
id	UUID PK	
tenant_id	UUID FK → tenants.id	RLS key — one set per tenant
name	TEXT NOT NULL	e.g. fee_reminder, payment_confirmation
body	TEXT NOT NULL	Template with {placeholder} tokens

sms_templates		
Column	Type / Constraint	Notes
updated_at	TIMESTAMPTZ	

9.3 Send Flow

- Admin clicks 'Send SMS' on fee dashboard → selects students (or bulk-select defaulters).
- API resolves template placeholders for each recipient.
- API calls SMS gateway (MSG91 preferred for India); logs each send in sms_logs.
- Gateway delivers a webhook callback with delivery status; API updates sms_logs.status.
- Tenant SMS credits decremented by count of successful sends.

9.4 API Routes

Method	Route	Description
POST	/api/sms/send	Send manual SMS to one or many guardians
GET	/api/sms/logs	List SMS logs for tenant with filters
POST	/api/sms/webhook	Gateway delivery callback (public, HMAC-verified)
GET	/api/sms/templates	List editable templates
PATCH	/api/sms/templates/[id]	Update a template

Phase 2 Extension

Automated SMS triggers (5 days before due, on due date, 3 days after) are implemented by adding a cron job that queries fee_ledger for upcoming/overdue entries and calls /api/sms/send. No schema changes required — this module is already designed for it.

10. Non-Functional Requirements

10.1 Performance Targets

Metric	Target	How
Page load (TTI)	< 2 seconds	Next.js SSR + Vercel edge; image optimisation
API response (p95)	< 400 ms	Indexed queries; Supabase connection pooling
Calendar render	< 1 second	Batch occurrence computation on server; JSON returned
Receipt PDF generation	< 3 seconds	Server-side HTML→PDF; async + signed URL
SMS send latency	< 5 seconds	Gateway async; UI shows 'sending' state

10.2 Security

- RLS on every table — tenant isolation enforced at database layer.
- JWT validated server-side on every API route handler.
- Role-checked middleware prevents route access mismatches.
- Supabase Storage: receipts and photos accessed via signed URLs (7-day expiry).
- SMS webhook endpoint HMAC-verified to prevent spoofed delivery callbacks.
- Rate limiting on auth routes (Supabase built-in) and SMS send endpoints.

10.3 Compliance

- DPDP Act (India): guardian consent logged at enrolment; SMS opt-out respected.
- Data export: admin can trigger full tenant data export (students, fees, schedules) as CSV.
- Data deletion: tenant admin can request account deletion; cascades a GDPR-compliant erasure workflow.
- GST: all receipts include GST amount; receipts carry a sequential receipt number per tenant.

10.4 Scalability

- Supabase free tier handles dev + early production. Scale to Pro (\$25/mo) when > 500 daily active users.
- Vercel scales automatically; no configuration needed for horizontal scale.
- Tenant isolation via RLS means adding tenants has zero performance impact on existing tenants.
- SMS gateway is stateless and swappable — add Exotel or Gupshup by changing one environment variable.

11. Environment & CI/CD

11.1 Environment Variables

Variable	Used In	Notes
NEXT_PUBLIC_SUPABASE_URL	Client + Server	Supabase project URL
NEXT_PUBLIC_SUPABASE_ANON_KEY	Client	Public anon key (safe to expose)
SUPABASE_SERVICE_ROLE_KEY	Server only	Admin key — never sent to client
SMS_GATEWAY_API_KEY	Server only	MSG91 / Twilio key
SMS_GATEWAY_SENDER_ID	Server only	Registered sender ID
SMS_WEBHOOK_SECRET	Server only	HMAC secret for delivery callbacks

11.2 Deployment Pipeline

- main branch → Vercel Production (auto-deploy on merge).
- develop branch → Vercel Preview (auto-deploy on push; unique preview URL per PR).
- Supabase migrations: tracked in supabase/migrations/ folder; applied via Supabase CLI (npx supabase db push).
- GitHub Actions: lint + type-check on every PR. Merge blocked if either fails.
- Supabase local dev: npx supabase start spins up local Postgres + Auth + Storage + Studio.

11.3 Folder Structure

Recommended Next.js App Router project structure:

- app/ — all routes (auth, admin, coach)
- app/api/ — Route Handlers (server-side API)
- components/ — shared UI components (shadcn/ui wrappers)
- lib/ — Supabase client, SMS gateway wrapper, PDF generator, utils
- lib/supabase/ — server.ts (service role), client.ts (anon), types.ts (generated)
- lib/sms/ — gateway adapter (pluggable)
- lib/pdf/ — receipt generator
- middleware.ts — auth + role + tenant routing
- supabase/migrations/ — SQL migration files
- supabase/seed.sql — dev seed data

12. Phase 2 — Design Decisions Made in v1.0

The following v1.0 design decisions exist specifically to avoid rewrites when Phase 2 features are added. Each row describes what is built now and why.

Phase 2 Feature	v1.0 Decision That Enables It	Schema / Code Impact
Automated SMS Triggers	sms_templates + sms_logs tables built in v1.0	Add cron job calling existing /api/sms/send
Attendance Tracking	sessions table exists; batch_students has status	Add attendance column to sessions; no schema change
Parent Portal	guardian_email stored; profiles.role = 'parent' supported	Add auth flow for parent invite email
In-App Notifications	notification_type enum reserved in profiles	Add Supabase Realtime subscription on client
Expense Tracking	tenants.currency + paise model ready	Add expenses table; same paise convention
Finance P&L	fee_payments + expense tables feed the same queries	Add aggregation queries; no model change
Receipt Auto-Email	guardian_email stored; receipt_url on payments	Add Supabase Edge Function trigger on insert
CSV Bulk Import	Import endpoint /api/students/import built in v1.0	Extend template with attendance history

Engineering Principle

Build v1.0 lean — but design every database table, API contract, and component as if v3.1 is coming. The cost of redesigning a fee model or role system mid-product is 10x the cost of designing it right the first time.

13. Feature Build Checklist

Use this checklist to track progress module by module. Each item is independently deployable and testable.

Module 1 — Foundation & Auth

- Supabase project created, environment variables set
- tenants and profiles tables with RLS policies
- Next.js project scaffolded with TypeScript + Tailwind
- Login / Signup / Invite pages built
- middleware.ts: JWT → role → route enforcement
- Supabase Storage buckets created (logos, photos, receipts)
- Subdomain routing middleware

Module 2 — Academy Setup

- Onboarding wizard (4 steps)
- Settings page: profile, working hours, fee settings, SMS settings
- Plan gating utility (planGuard)

Module 3 — Coach Management

- Coach CRUD: create, list, edit, deactivate
- Invite email via Supabase Auth
- Coach dashboard: calendar, batches, students

Module 4 — Student Management

- Student CRUD with all required + optional fields
- CSV import with validation + preview
- Student profile page (batch list, fee summary, jersey details)

Module 5 — Batch Management

- Batch CRUD
- Conflict detection (coach + venue)
- Capacity indicator + waiting list
- Batch lifecycle: Active / Paused / Closed

Module 6 — Calendar & Scheduling

- Day / Week / Month calendar views
- Batch occurrence generation algorithm
- 1-to-1 session booking with conflict check
- Drag-and-drop rescheduling (admin only)
- Session detail card

Module 7 — Fee Management

- Monthly ledger auto-generation (cron or manual trigger)
- Payment recording (full + partial)
- PDF receipt generation + Supabase Storage upload
- Fee dashboard: summary cards + defaulters list
- CSV / PDF export

Module 8 — SMS Notifications

- Manual SMS send from fee dashboard
- Template management with placeholder resolution
- Delivery webhook + status logging
- Credit balance tracking + low-credit alert